

Flow Matching ReSkill 流程

1. 实验 Setting

主要关注离线数据中 pick / push 技能比例变化对 ReSkill 类方法的影响。数据集命名为：

```
1 | fetch_block_push{push}_pick{pick}
```

现在构造了：

```
1 | fetch_block_push999_pick1
2 | fetch_block_push500_pick1
3 | fetch_block_push300_pick1
4 | fetch_block_push200_pick1
5 | fetch_block_push100_pick1
6 | fetch_block_push50_pick1
7 |
8 | fetch_block_push1_pick50
9 | fetch_block_push1_pick100
10 | fetch_block_push1_pick200
11 | fetch_block_push1_pick300
12 | fetch_block_push1_pick500
13 | fetch_block_push1_pick999
```

思路是构造不同 pick / push 比例，观察原 RNVP ReSkill 在某些比例下是否明显退化，再比较 Flow Matching prior 是否能缓解这种长尾 skill prior 学习问题。

这里的 pick / push 只影响离线 skill 数据分布。在线 RL 阶段仍然在目标环境中交互，由高层 PPO 输出 n ，再通过 skill prior 生成 latent skill z 。

2. 总体流程

原 ReSkill 的在线结构是：

```
1 | state o
2 |   -> high-level PPO actor 输出 n
3 |   -> skill prior p(z | o, n) 采样 skill latent z
4 |   -> SkillVAE decoder 解码基础动作
5 |   -> residual PPO 修正动作
6 |   -> 环境交互
```

Flow_skill1_1 保持这个结构不变，只替换 skill prior，用流模型实现：

```
1 | RNVP prior:      p(z | o, n)
2 | Flow prior:      eta -> Euler flow -> z, condition = [o, n]
```

高层 PPO 的动作仍然是 n ，flow prior 只负责把 $[o, n]$ 映射到一个符合离线 skill 分布的 latent z 。

3. 离线阶段一：训练 SkillVAE

离线数据会被切成长度为 H 的 chunk：

$$\begin{aligned} S_t &= (s_t, \dots, s_{t+H-1}) \\ A_t &= (a_t, \dots, a_{t+H-1}) \end{aligned} \quad (1)$$

3.1 Skill Encoding

先在每个时间步拼接状态和动作：

$$x_i = [s_{t+i}, a_{t+i}] \quad (2)$$

然后整段序列输入 LSTM：

$$h = LSTM(x_0, \dots, x_{H-1}) \quad (3)$$

最后把 LSTM 的 hidden state 送入 encoder MLP，得到高斯后验参数：

$$\begin{aligned} \mu_z, \log_{var_z} &= \text{EncoderMLP}(h) \\ \sigma &= \sqrt{var} \end{aligned} \quad (4)$$

重参数采样：

$$\begin{aligned} z &= \mu_z + \sigma \cdot \epsilon \\ \epsilon &\sim N(0, I) \end{aligned} \quad (5)$$

这个 z 表示整个 chunk 的 skill。

3.2 Skill Decoding

decoder 是 closed-loop 的状态条件动作 decoder。同一个 z 会 repeat 到 chunk 的每个时间步：

$$z_{tiled} = \text{repeat}(z, H) \quad (6)$$

然后每一步用当前状态和同一个 latent skill 解码动作：

$$\hat{a}_{t+i} = \text{Decoder}(s_{t+i}, z) \quad (7)$$

对应代码：

```
1 decode_inputs = torch.cat((states, z_tiled), 2)
2 reconstruction = self.run_decode_batch(decode_inputs, self.decoder)
```

3.3 SkillVAE Loss

SkillVAE 的训练目标是动作重建损失加 KL 正则：

$$\mathcal{L}_{skill} = \mathcal{L}_{BC} + \beta \cdot \mathcal{L}_{KL} \quad (8)$$

其中代码里 `beta=1e-8`，KL 权重非常小，主要学习能重建动作 chunk 的 latent skill。

4. 离线阶段二：训练 Flow Matching Skill Prior

训练 SkillVAE 的同时，会用当前 batch 的 SkillVAE encoder 得到目标 latent skill：

```
1 output = self.skill_vae(data)
2 skill = output.z.detach()
```

这里的 `skill` 就是 flow prior 要拟合的目标：

$$z^* = \text{SkillVAEEncoder}(S_t, A_t) \quad (9)$$

4.1 Flow Prior 的条件

离线训练时没有高层 PPO 的 n ，所以沿用原 ReSkill 的思想：用**第一步动作**近似未来在线的高层动作。

代码中使用：

```
1 state = data["obs"][ :, 0, :]
2 action = data["actions"][ :, 0, :] / 2.
3 action = action_ori + 0.2 * torch.normal(0, 1, action.shape)
4 condition = torch.cat([state, action], dim=1)
```

因此离线 flow prior 的条件是：

$$c = [s_t, a_t/2 + noise] \quad (10)$$

在线时则替换为：

$$c = [o_t, n_t] \quad (11)$$

这里 n_t 是高层 PPO 输出的连续向量，维度和 action 维度一致。

4.2 Flow Matching Objective

Flow prior 学的是从标准高斯噪声到 SkillVAE latent 的条件速度场。

采样：

$$\begin{aligned} x_0 &= \eta_z \sim \mathcal{N}(0, I_{d_z}) \\ x_1 &= z^* \\ t &\sim U(0, 1) \end{aligned} \quad (24)$$

构造线性插值：

$$x_t = (1 - t) * x_0 + t * x_1 \quad (25)$$

目标速度是：

$$u_t = x_1 - x_0 = z^* - \eta_z \quad (26)$$

flow matching速度场学习：

$$v_\theta(x_t, t, c) = z^* - \eta_z \quad (27)$$

损失为：

$$\mathcal{L}_{flow} = ||v_\theta(x_t, t, c) - (z^* - \eta_z)||^2 \quad (28)$$

4.3 Flow matching Euler Sampling

训练好速度场后，采样时从高斯噪声开始：

$$z_0 = \eta_z \quad (29)$$

用 Euler 积分推进：

$$z_{k+1} = z_k + dt * v_\theta(z_k, t_k, c) \quad (30)$$

最后得到：

$$z = z_K \quad (31)$$

flow_steps=10。

5. 在线阶段：Flow ReSkill 训练

在线阶段发生真实环境交互。每个 high-level step 先选择一个 latent skill，然后执行 H 个环境步。

5.1 高层 PPO 选择 n

当前状态为：

$$o_t \quad (32)$$

高层 PPO actor 输出：

$$n_t = \pi_{high}(o_t) \quad (33)$$

对应代码：

```
1 | n, v_agent, logp_agent, mu, std = agent.ac.step(o)
```

这里 n_t 不是最终环境动作，而是 skill prior 的条件变量。

5.2 Flow Prior 生成 z

在线采样时构造：

$$c_t = [o_t, n_t] \quad (34)$$

然后通过 flow prior 从噪声生成 latent skill：

$$\begin{aligned} \eta_z &\sim \mathcal{N}(0, I_{d_z}) \\ z_t &= \text{FlowPrior}(c_t, \eta_z) \end{aligned} \quad (35)$$

代码：

```
1 | cond = torch.cat((o, n), dim=1)
2 | z = skill_prior.sample_z_torch(cond)
```

5.3 Skill Decoder 执行 H 步

得到 z_t 后，在接下来 H 个环境步里复用同一个 z_t 。

每个环境步：

$$a_{dec} = \text{SkillVAE_Decoder}(o_{t+i}, z_t) \quad (36)$$

代码：

```
1 | obs_z = torch.cat((o2, z), 1)
2 | a_dec = skill_vae.decoder(obs_z)
```

5.4 Residual Policy 修正动作

Residual PPO 的输入是：

$$[o_{t+i}, z_t, a_{dec}] \quad (37)$$

输出 residual action：

$$a_{res} = \pi_{res}([o_{t+i}, z_t, a_{dec}]) \quad (38)$$

最终环境动作：

$$a = a_{dec} + \text{residual_factor} * a_{res} \quad (39)$$

其中 `residual_factor` 随训练步数逐渐增大，让训练前期更依赖 SkillVAE decoder，后期允许 residual policy 做更多修正。

对应代码：

```
1 reskill/train_reskill_agent_res.py
```

6. 在线更新

6.1 高层 PPO 更新

高层 PPO 的 action 是 n_t 。

执行一个 skill chunk 后，累计该 chunk 内的环境 reward：

$$R_{skill} = \sum_{i=0}^{H-1} r_{t+i} \quad (40)$$

高层 PPO 用这个 skill-level reward 更新：

$$\pi_{high}(o_t) - > n_t \quad (41)$$

它学习的是：什么样的 n_t 经过 flow prior 后更容易生成有用的 skill latent z_t 。

6.2 Residual PPO 更新

Residual PPO 是低层修正策略，按环境单步收集数据。

它的 observation 是：

$$[o_{t+i}, z_t, a_{dec}] \quad (42)$$

它的 action 是：

$$a_{res} \quad (43)$$

它的 reward 是环境单步 reward：

$$r_{t+i} \quad (44)$$

因此高层 PPO 和 residual PPO 的时间尺度不同：

- 1 high-level PPO: skill chunk 级别
- 2 residual PPO: 环境 step 级别

7. Q-Guidance Flow Sampling

当前版本还支持在 flow teacher 的 Euler 采样过程中加入 Q-guidance。

启用条件：

```
1 prior_model = Flow
2 use_grad = 1
3 guidance_scale > 0
4 use_student = 0
```

代码中会训练一个 chunk critic：

$$Q(o, z) \quad (45)$$

它评估当前状态下执行 latent skill z 的 chunk return。

在 flow Euler 采样时，不再只用速度场：

$$z_{k+1} = z_k + dt * v_{\theta}(z_k, t_k, c) \quad (46)$$

而是加入 Q 对 latent 的梯度：

$$z_{k+1} = z_k + dt * (v_{\theta}(z_k, t_k, c) + \text{guidance_scale} * \nabla_z Q(o, z_k)) \quad (47)$$

直观含义是：flow prior 负责保证 z 像离线 skill 分布，Q-gradient 负责把采样方向稍微推向在线 critic 认为更高价值的 skill。

8. 和 RNVP ReSkill 的区别

RNVP ReSkill：

```
1 o -> high-level PPO -> n
2 [o, n] -> RNVP prior -> z
3 [o, z] -> SkillVAE decoder -> a_dec
4 [o, z, a_dec] -> residual PPO -> a_res
5 a = a_dec + residual * a_res
```

Flow Matching ReSkill：

```
1 o -> high-level PPO -> n
2 [o, n], eta_z -> flow matching prior -> z
3 [o, z] -> SkillVAE decoder -> a_dec
4 [o, z, a_dec] -> residual PPO -> a_res
5 a = a_dec + residual * a_res
```

所以核心差异只有 skill prior 的建模方式：

```
1 RNVP：显式可逆密度模型
2 Flow：条件速度场 + Euler sampling
```

高层 PPO、SkillVAE decoder、residual PPO 的整体在线框架保持一致。